Programozáselmélet
1.gyakorlat

Bevezetés:

Célunk olyan módszerek vizsgálata, amelyek réven egy feladathoz garantáltan (matematikai pontossággal igazoltan) helyes program, azaz a feladatot biztosan megoldó program készíthető. (Ilyen módszer például a visszavezetés.)

Ehhez egy olyan matematikai modellre van szükségünk, amellyel le tudjuk írni a programozási fogalmakat (feladat, program, megoldás). A programozáselmélet kurzuson ezt a modellt mutatjuk be, és a rákövetkező programozási módszertan kurzuson a modellre épülő módszerekről lesz szól.

1.feladat: Modellezzük az alábbi feladatot!

„Számoljuk ki egy egyenes vonalú egyenletes mozgás sebességét, ha ismert a megtett út hossza, és az eltelt idő!”

adat + kapcsolatok

adat ~ név, pillanatnyi érték, mértékegység, típus (lehetséges értékek halmaza és az azokkal végezhető műveletek), ---> típusérték-halmaz

kapcsolat ~ reláció

a) Input-output alapú szemlélet

input adat ~ ismert, adott értékű adat

A feladat input adatai: út és idő, amelyek lehetséges értékei valós számok;

az összes input adat lehetséges értékei: $\mathbb{R} \times \mathbb{R}$

output adat ~ kiszámolandó értékű adat

A feladat output adata: sebesség, amely lehetséges értékei valós számok;

az összes (egyetlen) output adat lehetséges értéke: $\mathbb{R}$

A feladat egy leképezés, amely inputhoz outputot rendel:

$(120.0, 2.0) \longrightarrow 60.0$

$(270.0, 1.5) \longrightarrow 180.0$

...

Összefoglalva:

$F : \mathbb{R} \times \mathbb{R} \longrightarrow \mathbb{R}$

$\mathcal{D}_F = \mathbb{R}_0^+ \times \mathbb{R}^+$

$F(x, y) ::= x/y \quad (\forall (x, y) \in \mathcal{D}_F)$

b) Állapot alapú szemlélet

- Az input-output alapú szemlélettel nem lehet modellezni azt, hogy mi történjen a feladatot megoldó program input változóival (őrizzék-e kezdő értékeiket vagy ne). Nem lehet megmondani azt, hogy egy egyszerre input- és output változót szeretnénk használni (pl: helyben rendezés egy tömbben).
- A programok modellezésénél természetesebb, ha a program különböző állapotairól beszélhetünk.

állapot = összes (input, output és egyéb) adat egy lehetséges érték-együttese

A feladat állapotai: út, idő, sebesség értékeket mutató valósszám hármasok;

állapottér = az összes állapot

A feladat összes lehetséges állapota: $\mathbb{R} \times \mathbb{R} \times \mathbb{R}$

A feladat egy leképezés, amely csak az $\mathbb{R}_0^+ \times \mathbb{R}^+ \times \mathbb{R}$ kezdő állapotokban értelmezett, és ezekhez rendel célállapotot:

$$(120.0, 2.0, *) \longrightarrow (120.0, 2.0, 60.0)$$

$$(270.0, 1.5, *) \longrightarrow (270.0, 1.5, 180.0)$$

…

A Descart szorzat nem elég pontos matematikai fogalom az állapottér modellezésére (de még az input- illetve output adatok modellezésére sem). Több azonos típusú adat esetén ($\mathbb{R} \times \mathbb{R} \times \mathbb{R}$) nem egyértelmű, hogy egy komponens melyik adatot jelöli. A jelentésében két különböző Descart szorzat ($\mathbb{N} \times \mathbb{R}$ és $\mathbb{R} \times \mathbb{N}$) pedig képes ugyanazt (ugyanazon adatokból álló) állapotteret jelölni.

Ezért az állapottér jelölésében egyedi címkék segítségével kell megkülönböztetnünk az adatokat. Ha a feladatban az s, t, v címkékkel rendre az út, idő, sebesség adatot azonosítják, akkor az állapotteret az $(s : \mathbb{R}, t : \mathbb{R}, v : \mathbb{R})$ szimbólum jelölheti. A címkéket hívjuk majd változóknak.

A feladat egy állapotát címkézett értékek halmazaként adhatjuk meg: $\{s : x, t : y, v : z\}$ ahol $(x, y, z \in \mathbb{R})$, amely megmutatja, hogy az aktuális állapotban a változók éppen milyen értéket vesznek fel. Ugyanakkor ez a jelölés rövidíthető az (x, y, z) érték-hármassal, ha előzetesen rögzítjük az (s, t, v) sorrendet az adatok között.

Összefoglalva:

$$F : (s : \mathbb{R}, t : \mathbb{R}, v : \mathbb{R}) \longrightarrow (s : \mathbb{R}, t : \mathbb{R}, v : \mathbb{R})$$

$$\mathcal{D}_F = (s : \mathbb{R}_0^+, t : \mathbb{R}^+, v : \mathbb{R})$$

$$\forall (x, y, z) \in \mathcal{D}_F: F(x, y, z) ::= (x, y, x/y) \quad \text{vagy egyszerűbben} \quad F(x, y, *) ::= (x, y, x/y)$$

1.kvíz kérdés: Mi az állapottere az alábbi feladatnak: „Oszt-e egy adott természetes szám egy másik adott természetes számot? Adjuk meg a feladat egy lehetséges kezdő- és célállapot párját is!

2.feladat: Modellezzük az alábbi feladatot!

„Adjuk meg egy természetes szám egy természetes osztóját!”

$A = (n : \mathbb{N}, d : \mathbb{N}),\ D_F = A$

$F = \{ (6,*) \rightarrow (6,1)$

$(6,2)$

$(6,3)$

$(6,6)$

$\ldots \quad \}$

feladat = egy nem-determinisztikus leképezés az állapottérből az állapotérre, egy homogén binér reláció.

$F \subseteq A \times A, \qquad D_F = A$

$F(x, y) = \{ (u, v) \in A \mid u = x \wedge v \mid x \} \qquad (\forall (x, y) \in \mathcal{D}_F)$

rövidebben: $F(x, y) = \{ (u, v) \mid u = x \wedge v \mid x \}$

egyszerűbben $F(x, *) = \{ (u, v) \mid u = x \wedge v \mid x \}$

A feladat célállapotának nem muszáj megőriznie az input adatok kezdőértékeit. Általában ez a megoldó programon múlik, kivéve, ha a feladat kitűzői kifejezetten előírják az input adatok megőrzését. Formálisan természetesen különbözik a feladat leírása, ha nem kell az input adatokat megőrizni (még nem-determinisztikusabb lesz)

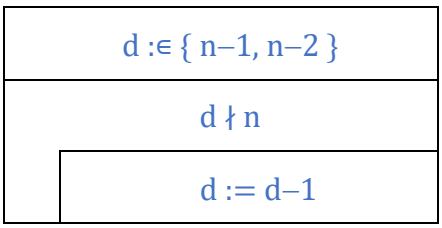
$F \subseteq A \times A, \qquad D_F = A$

$F(x, y) = \{ (u, v) \mid v \mid x \} \quad \forall (x, y) \in \mathcal{D}_F$ vagy egyszerűbben $F(x, *) = \{ (*, v) \mid v \mid x \}$

2.kvíz kérdés: Modellezzük az alábbi feladatot: „Adott két egész típusú változó. Cseréljük ki az értékeiket!”

3.feladat: Adjuk meg az alábbi program néhány végrehajtását!

$A = (n : \mathbb{N}, d : \mathbb{N})$



Hogyan változik a változók értéke az alábbi állapotokból indulva?

$(6, 8) \longrightarrow < (6, 8), (6, 5), (6, 4), (6, 3) >$

$(3, 8) \longrightarrow < (3, 8), (3, 2), (3, 1) >$

$(2, 8) \longrightarrow < (2, 8), (2, 1) >$

$(1, 8) \longrightarrow < (1, 8), (1, 0), \text{fail} >$

$(0, 8) \longrightarrow < (0, 8), \text{fail} >$

$(6, 8) \longrightarrow < (6, 8), (6, 4), (6, 3) >$

$(3, 8) \longrightarrow < (3, 8), (3, 1) >$

$(2, 8) \longrightarrow < (2, 8), (2, 0), \text{fail} >$

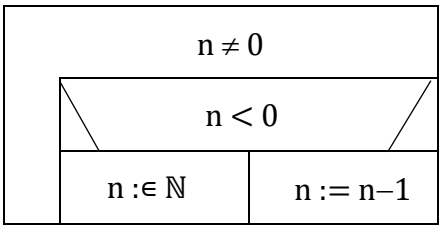
$(1, 8) \longrightarrow < (1, 8), \text{fail} >$

program = leképezés, amely megadja bármelyik kiinduló állapotra az abból induló végrehajtásokat, azaz állapot sorozatokat

Milyen tulajdonságai vannak a végrehajtásoknak?

- véges (normális vagy hibás), vagy végtelen hosszúak
- minden állapotból indul legalább egy
- egy állapotból több végrehajtás is indulhat
- első állapota mindig a kiinduló állapot

3.kvíz kérdés: Adjuk meg, hogy a mellékelt program milyen végrehajtásokat indít a 4, a 0, és a -5 állapotokból indulva!



4.feladat: Adjuk meg az alábbi program néhány végrehajtását!

$A = (x : \mathbb{Z}, y : \mathbb{Z}, z : \mathbb{Z})$

$z := x$
$x := y$
$y := z$

```
void swap(int &x, int &y) {
    int z = x;
    x = y;
    y = z;
}
```

Önmagában egy programnak nincsenek input- és output változói: csak változói vannak. Csak amikor adott a feladat, amelyet a programmal meg akarunk oldani, akkor válik világossá, hogy a programot tartalmazó környezet mely változóknak kell kezdő értéket adnia a program elindulása előtt, és mely változók értékét kell kiolvasnia a program leállása után, azaz mik lesznek az input- és output változók, amelyek a program egy lehetséges alapállapotterét alkotják.

Legyen most megállapodás szerint az alapállapottér: $A = (x : \mathbb{Z}, y : \mathbb{Z})$, következésképpen pedig a $z : \mathbb{Z}$ egy segédváltozó, amely kezdetben még nem létezik, és legkésőbb a program leállásával megszűnik.

Hogyan változik a változók értéke az $\{x : -4, y : 8\}$ állapotból indulva?

$\{x : -4, y : 8\} \longrightarrow <\{x : -4, y : 8\}, \quad \{x : -4, y : 8, z : -4\}, \quad \{x : 8, y : 8, z : -4\},$
$\{x : 8, y : -4, z : -4\}, \quad \{x : 8, y : -4\} \quad >$

Milyen tulajdonságai vannak még a végrehajtásoknak?

- állapotai vagy az alap-állapottérhez, vagy az annak segéd-változókkal kiegészített állapotteréhez tartoznak
- mindig alap-állapottérbeli állapotból indulnak
- ha végesek és nem hibásak, akkor alap-állapottérbeli állapotban állnak meg
- A segédváltozók keletkezését és megszűnését nem kell önálló állapot-változásként feltüntetni